



Museum Heist

Grid-based stealth game

Greedy · Backtracking · Linked List · BST



Agenda

 Introduction & goal

 System architecture

 C++ engine & JSON bridge

 Data structures

 Algorithms (Greedy / BT)

 Interface & mechanics

 Testing & validation

 Conclusions



Tech stack:

C++

Python

JSON

Pygame

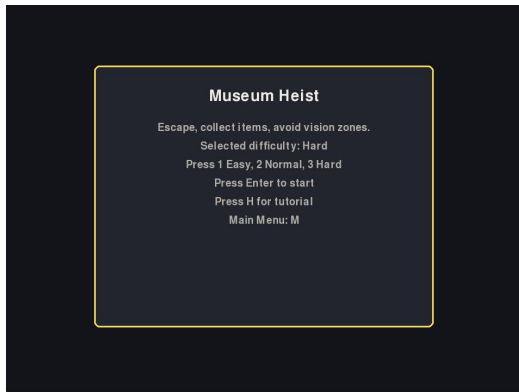
What is Museum Heist?

Concept

A grid-based stealth game where **every game mechanic is a course algorithm or data structure**.

- Navigate a museum grid and **collect valuable items**
- Avoid **cameras, guards** and orange vision zones
- Reach the **green exit** before losing by alarm, movement limit, or time limit

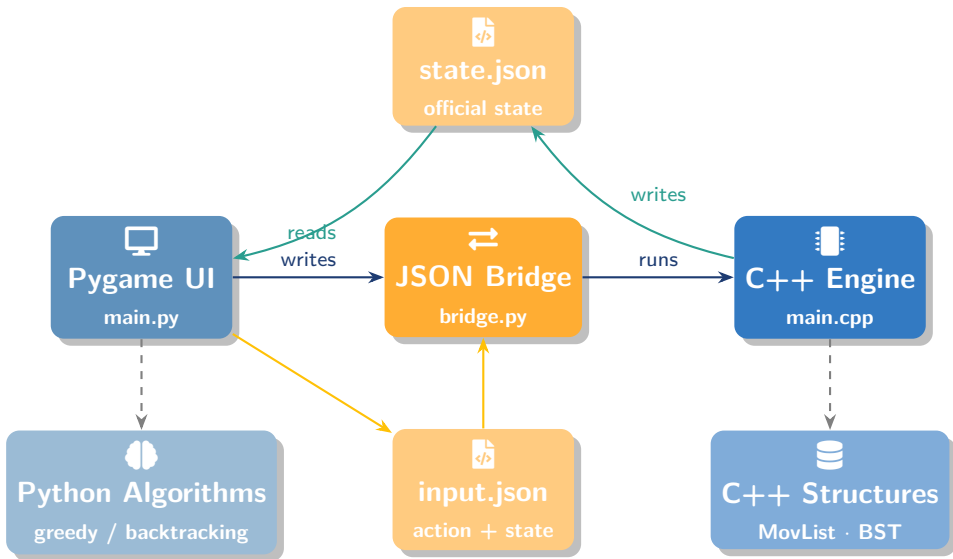
Motivation



Main menu — difficulty selection



System Architecture





Execution Cycle



Input: Pygame detects key press
(WASD / arrows)



Write: Python writes action + previous state to `input.json`



C++ validation: engine checks bounds, walls, vision zones, items; writes `state.json`



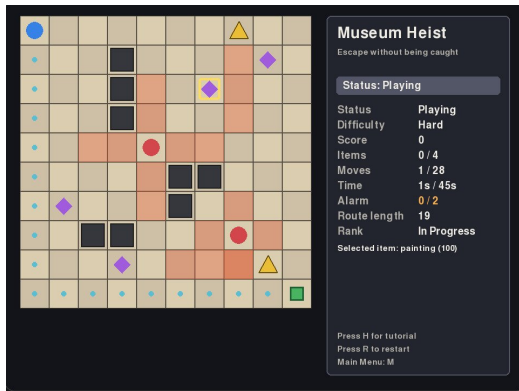
Read: Python loads the new state



Recompute: greedy and backtracking recalculate



Render: Pygame redraws board + HUD



Live gameplay — board + sidebar HUD

Parameters by difficulty

Level	Grid	Alarm	Moves/Time
Easy	8×8	4	45 / 90s
Normal	8×8	3	35 / 60s
Hard	10×10	2	28 / 45s

- Compiled with `g++ -std=c++17`
- Validates: bounds, walls, cameras, guards, vision zones
- Wall collision $\Rightarrow$ **alarm++**
- Entering vision zone $\Rightarrow$ **Game Over**

Listing 1: Movement validation (simplified)

```
1 // Check destination cell
2 if (outOfBounds(nr, nc))
3     return; // no change
4
5 if (grid[nr][nc] == WALL) {
6     alarm++; // wall hit
7     return;
8 }
9
10 if (isVisionZone(nr, nc)
11     || isCamera(nr, nc)
12     || isGuard(nr, nc)) {
13     game_over = true;
14     return;
15 }
16 // Valid move
17 player = {nr, nc};
18 history.push_back(nr, nc);
```

⇄ JSON Bridge

📄 input.json (Python → C++)

```
{  
  "action": "move",  
  "direction": "right",  
  "difficulty": "Normal",  
  "player": {"row":0,"col":0},  
  "score": 0,  
  "alarm": 0,  
  "elapsed_seconds": 12  
}
```

📄 state.json (C++ → Python)

```
{  
  "grid": [...],  
  "player": {"row":0,"col":1},  
  "vision_zones": [...],  
  "items_sorted_by_value": [...],  
  "movement_history": [...],  
  "alarm": 0,  
  "status": "playing",  
  "rank": "In Progress"  
}
```

- ✓ JSON is human-readable, easy to debug, and available in Python via the standard `json` module — no extra dependencies.

 The project does **NOT** use `std::list`, `std::map`, or `std::set` for the required structures — both are **implemented from scratch**.

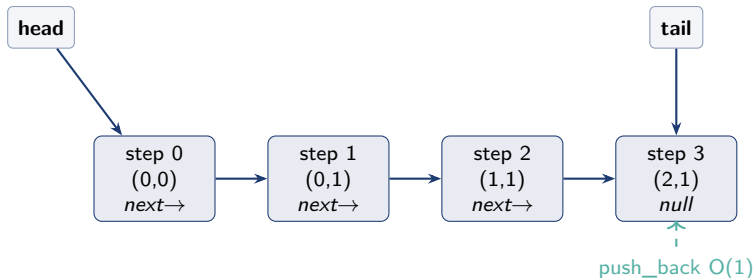
MovementList — Linked List

- Each node stores: `row`, `col`, `step`, `*next`
- tail pointer → **append $O(1)$**
- Traverse to write JSON: $O(n)$
- Clear all nodes: $O(n)$
- Records complete **movement history**

ItemBST — Binary Search Tree

- Orders items by **value**
- Left → smaller Right → larger
- Tie-breaker: `item_id`
- Insert: $O(\log n)$ avg / $O(n)$ worst case
- Inorder → `items_sorted_by_value`

MovementList — Diagram

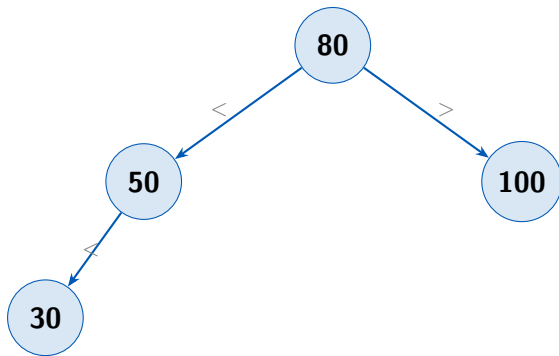


- **push_back**: always at end via tail → $O(1)$
- **Traverse**: serialize movement history → $O(n)$
- **Clear**: free all nodes → $O(n)$
- No `std::list` used — custom

 The `tail` pointer is the key optimization: appending is always constant time regardless of list length.



ItemBST — Diagram



Inorder traversal (ascending): **30 → 50 → 80 → 100**
Written as `items_sorted_by_value` in `state.json`

Insert: $O(\log n)$ avg / $O(n)$ worst (unbalanced) — Traversal & Clear: $O(n)$. No `std::map` used.

Greedy Algorithm — Item Selection

Decision logic (priority order)

- 1 **Highest-value** item near a guard and **outside** a vision zone
- 2 Otherwise: highest-value item that is **completely safe**
- 3 If all are dangerous: highest-value with a **warning**

Key property

One **local decision** per turn. No future route exploration.

Not globally optimal — but fast.

Listing 2: greedy.py (pseudocode)

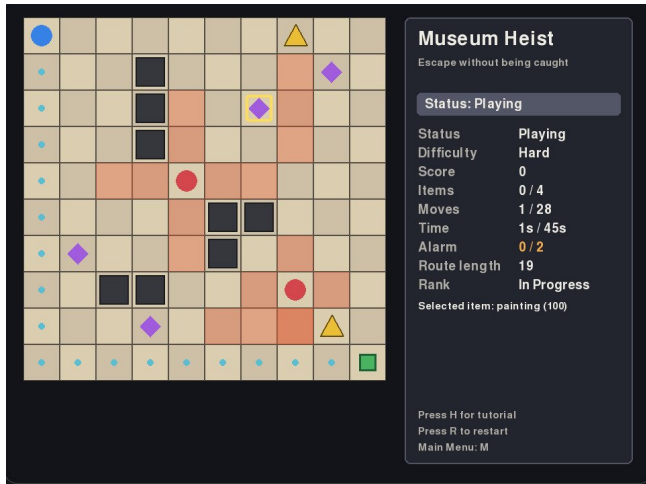
```
def choose_greedy_item(state):
    items = remaining_items(state)
    if not items:
        return None

    # 1. Near guard, safe
    near_guard = [i for i in items
                  if near_guard(i) and safe(i)]
    if near_guard:
        return max(near_guard, key=val)

    # 2. Completely safe
    safe_items = [i for i in items
                  if safe(i)]
    if safe_items:
        return max(safe_items, key=val)

    # 3. Any item (warning)
    return max(items, key=val)
```

Greedy — Visual Output in Game



Sidebar shows:

Selected item: painting
(100)

The item with the highest value that is safe and near a guard is highlighted with a **yellow outline** on the board.

- ★ Yellow items = collectibles
- Orange zones = vision (danger)
- Red circles = guards

Structure: choose – explore –
unchoose

- ❶ Add current cell to route
- ❷ If it is the exit → **success**
- ❸ Explore neighbors by distance to exit
- ❹ If none works → **unchoose** and backtrack

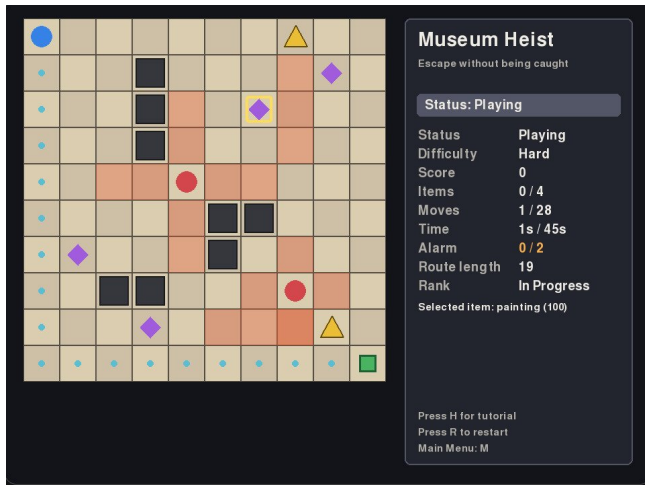
Invalid cells:

- Out of bounds
- Walls, cameras, guards
- Vision zones

Listing 3: backtracking.py (pseudocode)

```
1 def backtrack(row, col):
2     if invalid(row, col):
3         return None
4
5     route.append((row, col))
6     visited.add((row, col))
7
8     if is_exit(row, col):
9         return route.copy()
10
11     # Neighbors by distance
12     for nr, nc in neighbors(row, col):
13         result = backtrack(nr, nc)
14         if result:
15             return result
16
17     # Unchoose
18     route.pop()
19     visited.remove((row, col))
```

Backtracking — Route in Game



Route length: 19

The **cyan dots** on the board are the backtracking route cells from the player's current position to the exit.

- Route **avoids** walls, cameras, guards, and vision zones
- **Recalculated** after every single move
- If no safe path exists: route is empty
- The green square



Algorithm Comparison

Aspect	Greedy	Backtracking
Goal	Choose one item	Find route to exit
Decision	Local / immediate	Recursive / global
Guarantee	Local optimum	Feasible route if it exists
Complexity	$\mathcal{O}(i(g + v))$	$\mathcal{O}(4^n)$ worst case
In practice	Very fast	Bounded by map size
Language	Python	Python
File	greedy.py	backtracking.py

Both algorithms are **recomputed every turn** and their output is **rendered directly** on the board and in the sidebar HUD.



Interface & Mechanics

Game screens

- **Main menu:** difficulty selection
- **Tutorial:** objective & objects explained
- **Playing:** board + sidebar HUD
- **Game Over:** caught screen with reason
- **Escaped:** final metrics and rank

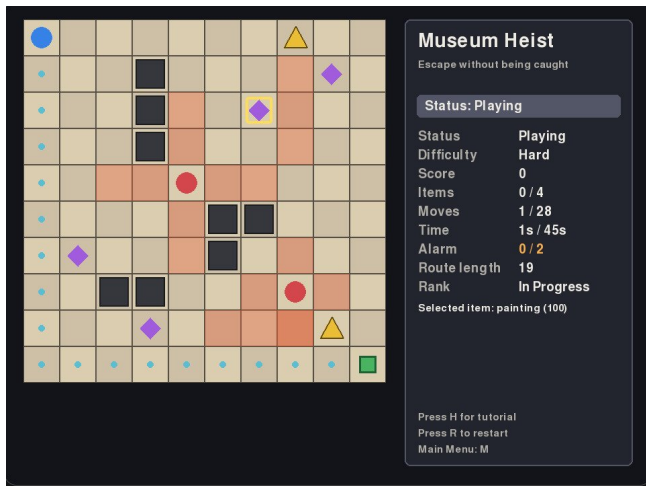
Controls

WASD / Arrows	Move player
R	Restart current difficulty

Rendering order

- 1 Floor (background)
- 2 Vision zones (orange)
- 3 Backtracking route (dots)
- 4 Walls (dark squares)
- 5 Items (diamonds/triangles)
- 6 Selected item outline
- 7 Exit (green square)
- 8 Cameras & guards
- 9 Player (top layer)

HUD — Real-Time Information



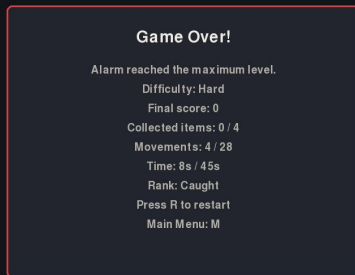
Live HUD fields:

Status	Playing
Difficulty	Hard
Score	0
Items	0 / 4
Moves	1 / 28
Alarm	0 / 2
Route length	19
Rank	In Progress

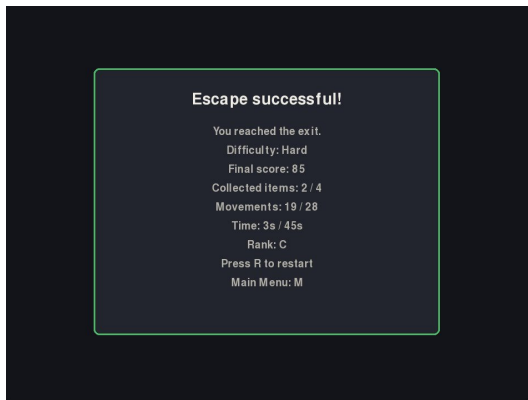
The **selected item** and reason are shown at the bottom of the sidebar (painting (100)).

✓ Testing & Validation

ID	Test	Expected
T01	Compilation	No errors
T03	Valid move	Player advances 1 cell
T06	Item collection	Score++ item disappears
T08	Vision zone	Immediate Game Over
T11	Time limit	Game Over by time
T12	Escape	Status = escaped
T15	Greedy	Item + reason returned
T16	Backtracking	Safe route or empty list



❖ Escape Screen — Final Metrics



Escape successful — final score and rank displayed

Metrics reported

- **Difficulty:** Hard
- **Final score:** 85 points
- **Collected:** 2 / 4 items
- **Movements:** 19 / 28 used
- **Time:** 3s / 45s
- **Rank:** C

Press **R** to restart or **M** to return to the main menu.



Limitations & Future Work

Current limitations

- Maps are **hardcoded** (no external files)
- Manual JSON parsing in C++
- BST can become **unbalanced**
- Backtracking optimizes feasibility, **not** score or alarm level

Future work

- Maps loaded from **external files**
- Guards with **dynamic patrol routes**
- Self-balancing BST (AVL / Red-Black)
- Route optimized by **score + alarm**
- Multiplayer support



Conclusions

- ✓ Course algorithms and data structures become **visible, playable mechanics**
- ✓ Architecture separates responsibilities: **C++** rules, **Python** visualization, **JSON** bridge
- ✓ All requirements fulfilled: greedy, backtracking, linked list, BST
- ✓ Three difficulty levels, win/loss conditions, final metrics, tutorial, and full controls



Museum Heist

Algorithms + Game = Learning



Universidad Distrital FJC
Computer Science I

2026-I



Thank You!

Questions & Discussion

